

Rapport de Travaux Pratiques : TP 1

Développement des systèmes distribués_python

Étudiante : AHLAME LAOUAD encadré par : Pr.Lhoussaine Daoudi

6 octobre 2026

Résumé

Ce rapport présente la résolution et les analyses techniques des différents exercices du TP 1 en Python. Les notions abordées couvrent les opérations de transtypage (*casting*), les structures conditionnelles avancées, les boucles d'itération et de contrôle, les fonctions avec paramètres par défaut, les listes ordonnées ainsi que la modélisation de structures de données pour les systèmes multi-agents.

1 Exercice 1 : Transtypage et Calcul de Moyenne Pondérée

1.1 Objectifs et Rappels

L'objectif est de maîtriser le transtypage (*type casting*). Toute entrée récupérée via `input()` est initialement de type chaîne de caractères (`str`). Afin d'effectuer des opérations arithmétiques, il est nécessaire de convertir ces données vers des types numériques appropriés :

- `float` pour les notes décimales ;
- `int` pour les coefficients entiers ;
- `str` pour formater la chaîne finale lors de l'affichage.

1.2 Implémentation Python

```
1 CNE = str(input("veuillez saisir votre identifiant étudiant:"))
2 note_projet = float(input("Veuillez saisir votre note du projet pratique :"))
3 note_ecrit = float(input("Veuillez saisir votre note d'examen écrit:"))
4
5 coeff_projet = int(input("Veuillez saisir les coefficient de votre TP :"))
6 coeff_ecrit = int(input("Veuillez saisir votre coefficient d'examen écrit :"))
7
8 # Calcul de la moyenne pondérée
9 d1 = (note_projet * coeff_projet + note_ecrit * coeff_ecrit)
10 d2 = coeff_projet + coeff_ecrit
11 moyenne = d1 / d2
12
13 # Affichage formaté
14 moyenne_str = str(moyenne)
15 print(f"Pour l'etudiant : {CNE} , la moyenne finale du module est de : {
    moyenne_str}")
```

1.3 Preuve d'Exécution

```
# --- À VOUS DE JOUER ---
# Écrivez votre code ici :
#1.CNE
CNE = str(input("veuillez saisir votre identifiant étudiant:"))
#2_note examen tp
note_projet= int(input("Veuillez saisir votre note du projet pratique :"))
#conversion type note tp
note_projet=float(note_projet)
#note examen écrit
note_ecrit=int(input("Veuillez saisir votre note d'examen écrit:"))
#conversion de a note examen écrit
note_ecrit=float(note_ecrit)
#coefficiens des tp et écrit
coeff_projet=input("Veuillez saisir les coefficient de votre TP :")
coeff_ecrit=input("Veuillez saisir votre coefficient d'examen écrit :")
coeff_projet=int(coeff_projet)
coeff_ecrit=int(coeff_ecrit)
#4 calcule moyenne
d1= (note_projet * coeff_projet + note_ecrit * coeff_ecrit)
d2= coeff_projet + coeff_ecrit
moyenne = d1/d2
#5 conversion resultat final:
moyenne = str(moyenne)
print(f"Pour l'etudiant : {CNE} , la moyenne finale du module est de : {moyenne}")
```

[2] ✓ 424s Python

... Pour l'etudiant : C132091919 , la moyenne finale du module est de : 17.5

2 Exercice 2 : Structures Conditionnelles et Équation du Second Degré

2.1 Objectifs et Démarche

Cet exercice analyse les racines d'une équation polynomiale $ax^2 + bx + c = 0$ en respectant une séparation stricte des blocs logiques :

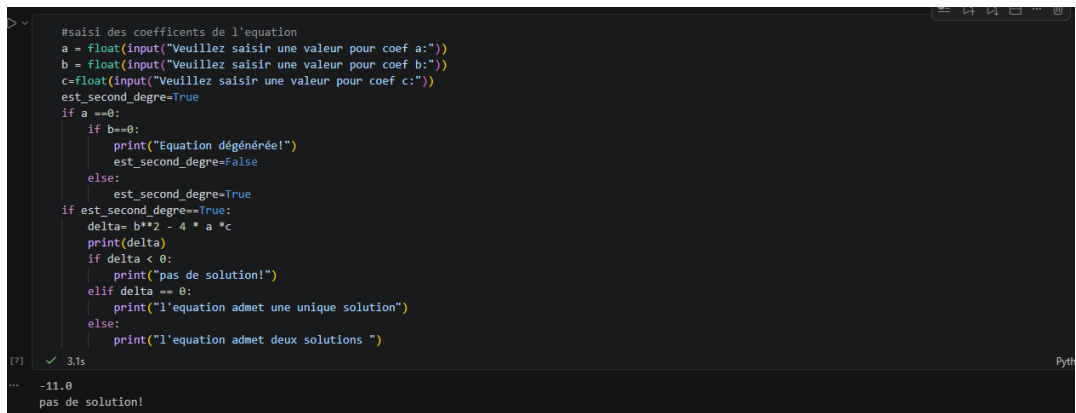
1. **Vérification du degré** : Identifier si l'équation est bien du second degré ($a \neq 0$). Si $a = 0$, un test imbriqué examine la nullité de b pour qualifier l'équation de dégénérée ou du premier degré.
2. **Résolution conditionnelle** : Si et seulement si `est_second_degre` est vrai, calculer le discriminant $\Delta = b^2 - 4ac$ et déterminer le nombre de solutions réelles.

2.2 Implémentation Python

```
1 a = float(input("Veuillez saisir une valeur pour coef a:"))
2 b = float(input("Veuillez saisir une valeur pour coef b:"))
3 c = float(input("Veuillez saisir une valeur pour coef c:"))
4
5 est_second_degre = True
6 if a == 0:
7     if b == 0:
8         print("Équation dégénérée !")
9         est_second_degre = False
10    else:
11        print("Équation du premier degré.")
12        est_second_degre = False
13 else:
14     est_second_degre = True
15
16 if est_second_degre:
17     delta = b**2 - 4 * a * c
18     print(f"Valeur de Delta : {delta}")
19     if delta < 0:
20         print("pas de solution!")
21     elif delta == 0:
22         print("l'equation admet une unique solution")
```

```
23 else:
24     print("l'equation admet deux solutions")
```

2.3 Preuve d'Exécution



```
#saisi des coefficients de l'equation
a = float(input("Veuillez saisir une valeur pour coef a:"))
b = float(input("Veuillez saisir une valeur pour coef b:"))
c = float(input("Veuillez saisir une valeur pour coef c:"))
est_second_degre=True
if a ==0:
    if b==0:
        print("Equation dégénérée!")
        est_second_degre=False
    else:
        est_second_degre=True
if est_second_degre==True:
    delta= b**2 - 4 * a *c
    print(delta)
    if delta < 0:
        print("pas de solution!")
    elif delta == 0:
        print("l'equation admet une unique solution")
    else:
        print("l'equation admet deux solutions ")
```

[7] ✓ 3.1s Pyth

... -11.0
pas de solution!

3 Exercice 3 : Contrôle de Saisie et Boucles (while et for)

3.1 Objectifs et Démarche

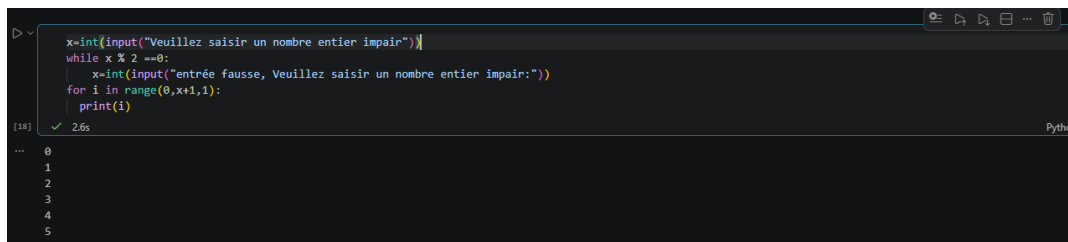
Le script demande la saisie d'un entier impair :

- Une boucle `while` filtre les entrées invalides (tant que $x \pmod{2} == 0$);
- Une boucle `for` combinée à `range(0, x + 1)` génère la suite des entiers jusqu'à x inclus.

3.2 Implémentation Python

```
1 x = int(input("Veuillez saisir un nombre entier impair : "))
2 while x % 2 == 0:
3     x = int(input("Entrée fausse, veuillez saisir un nombre entier impair : "))
4
5 for i in range(0, x + 1, 1):
6     print(i)
```

3.3 Preuve d'Exécution



```
x=int(input("Veuillez saisir un nombre entier impair:"))
while x % 2 ==0:
    x=int(input("entrée fausse, Veuillez saisir un nombre entier impair:"))
for i in range(0,x+1,1):
    print(i)
```

0
1
2
3
4
5

4 Exercice 4 : Fonctions, Immuabilité et Itérateurs

4.1 Objectifs et Analyse

Cet exercice met en lumière :

1. **L'immuabilité des types primitifs** : La variable passée en argument d'une fonction n'est pas modifiée en dehors du scope local de cette dernière.
2. **L'utilisation de `map()` et `zip()`** : Pour appliquer une fonction de transformation à chaque élément d'une liste et coupler des itérables.

4.2 Implémentation Python

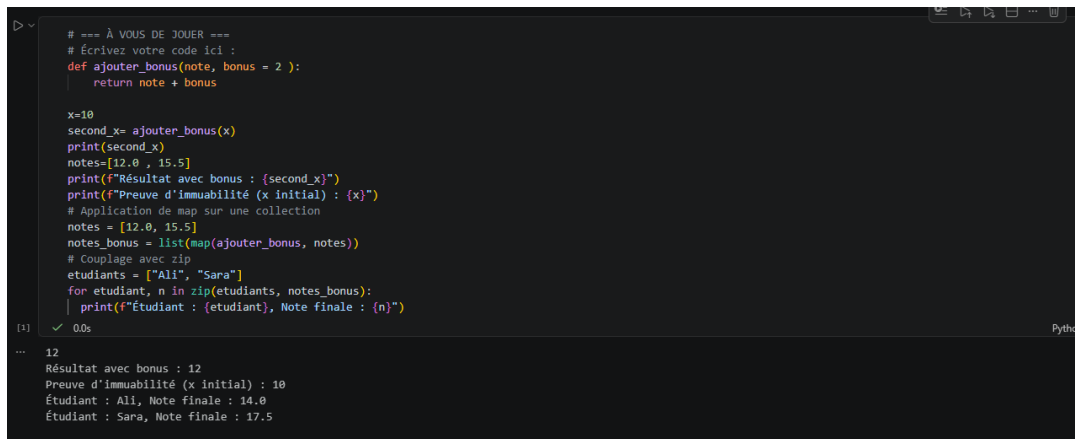
```
1 def ajouter_bonus(note, bonus=2):
2     return note + bonus
3
4 x = 10
5 second_x = ajouter_bonus(x)
6 print(f"Résultat avec bonus : {second_x}")
7 print(f"Preuve d'immuabilité (x initial) : {x}")
8
```

```

9 # Application de map sur une collection
10 notes = [12.0, 15.5]
11 notes_bonus = list(map(ajouter_bonus, notes))
12
13 # Couplage avec zip
14 etudiants = ["Ali", "Sara"]
15 for etudiant, n in zip(etudiants, notes_bonus):
16     print(f"Étudiant : {etudiant}, Note finale : {n}")

```

4.3 Preuve d'Exécution



```

# --- À VOUS DE JOUER ---
# Écrivez votre code ici :
def ajouter_bonus(note, bonus = 2 ):
    return note + bonus

x=10
second_x= ajouter_bonus(x)
print(second_x)
notes=[12.0 , 15.5]
print(f"Résultat avec bonus : {second_x}")
print(f"Preuve d'immuabilité (x initial) : {x}")
# Application de map sur une collection
notes = [12.0, 15.5]
notes_bonus = list(map(ajouter_bonus, notes))
# Couplage avec zip
etudiants = ["Ali", "Sara"]
for etudiant, n in zip(etudiants, notes_bonus):
    print(f"Étudiant : {etudiant}, Note finale : {n}")

```

12 Résultat avec bonus : 12
 Preuve d'immuabilité (x initial) : 10
 Étudiant : Ali, Note finale : 14.0
 Étudiant : Sara, Note finale : 17.5

5 Exercice 5 : Saisie Ordonnée et Calcul de la Médiane

5.1 Règles de Calcul de la Médiane

Pour une suite de n notes triées par ordre croissant :

- Si n est impair : la médiane est la valeur située à l'indice $\lfloor n/2 \rfloor$.
- Si n est pair : la médiane est la moyenne arithmétique des deux valeurs centrales :

$$\text{Médiane} = \frac{\text{notes}[(n/2) - 1] + \text{notes}[n/2]}{2}$$

5.2 Implémentation Python

```
1 total_notes = int(input("Veuillez saisir le nombre total de notes : "))
2 notes = []
3
4 for i in range(total_notes):
5     note = float(input(f"Saisissez la note {i + 1} : "))
6     while len(notes) > 0 and note < notes[-1]:
7         print("Erreur : la note doit être supérieure ou égale à la précédente !")
8         note = float(input(f"Ressaisissez la note {i + 1} (>= {notes[-1]}) : "))
9     notes.append(note)
10
11 print("\nListe des notes saisies :", notes)
12
13 n = len(notes)
14 if n % 2 == 1:
15     # Cas impair : l'élément du milieu
16     mediane = notes[n // 2]
17     print(f"Le nombre de notes est impair ({n}), la médiane est : {mediane}")
18 else:
19     # Cas pair : moyenne des deux éléments centraux
20     element_gauche = notes[(n // 2) - 1]
21     element_droit = notes[n // 2]
22     mediane = (element_gauche + element_droit) / 2
23     print(f"Le nombre de notes est pair ({n}), la médiane est : {mediane}")
```

5.3 Preuve d'Exécution

```
#1_nombre totl de notes
total_notes= int(input("Veuillez saisir le nombre total de nombre que vous souhaitez traiter:"))
notes=[]
#2 boucle pour saisir chaque note
for i in range(total_notes):
    note = float(input(f"Saisissez la note {i + 1} : "))
    while len(notes) > 0 and note < notes[-1]:
        print("Erreur : la note doit être supérieure ou égale à la précédente !")
        note = float(input(f"Ressaisissez la note {i + 1} (>= {notes[-1]}) : "))
    notes.append(note)

print("\nListe des notes saisies :", notes)
#3&4
n=len(notes)
if n % 2 == 1:
    # Cas impair : l'élément du milieu
    mediane = notes[n // 2]
    print(f"Le nombre de notes est impair ({n}), la médiane est : {mediane}")
else:
    # Cas pair : moyenne des deux éléments centraux
    element_gauche = notes[(n // 2) - 1]
    element_droit = notes[n // 2]
    mediane = (element_gauche + element_droit) / 2
    print(f"Le nombre de notes est pair ({n}), la médiane est : {mediane}")
```

✓ 122s Python

Liste des notes saisies : [1.0, 2.0, 3.0, 4.0]
Le nombre de notes est pair (4), la médiane est : 2.5

6 Exercice 6 : Modélisation Multi-Agents (IA) et Structures Imbriquées

6.1 Objectifs et Organisation des Données

Dans une architecture d'agents autonomes, la configuration combine :

- **Dictionnaires** : Pour indexer les agents et leurs hyperparamètres par clés textuelles ;
- **Listes** : Pour l'inventaire dynamique des capacités ;
- **Tuples immuables** : Pour associer de façon fixe un outil à sa version logicielle.

6.2 Implémentation Python

```
1 #1_création dictionnaire de configuration
2 config_ia={"role":"medical data extractor" , "modele": "gemini 2.5" , "
   temperature":0.2 , "memory_ACTIVE":True}
3 #2_ajout de clé
4 config_ia2={"agent_recherche":{
5     "modele":"gpt-4",
6     "temperature":0.2,
7     "outils": [("WebSearch", "v2.1"), ("Calculatrice", "v1.0")]
8 }}
9 #3 Modification de température
10 config_ia2["agent_recherche"]["temperature"]=0.5
11 #4 affichage de calculatrice
12 print(config_ia2["agent_recherche"]["outils"][1][0])
```

6.3 Preuve d'Exécution



```
#1_création dict
config_ia={"role":"medical data extractor" , "modele": "gemini 2.5" , "temperature":0.2 , "memory_ACTIVE":True}
#2_ajout de clé
config_ia2={"agent_recherche":{
    "modele":"gpt-4",
    "temperature":0.2,
    "outils": [("WebSearch", "v2.1"), ("Calculatrice", "v1.0")]
}}
#3
config_ia2["agent_recherche"]["temperature"]=0.5
#4
print(config_ia2["agent_recherche"]["outils"][1][0])
```

Calculatrice

7 Conclusion

Ce travail pratique a permis de mettre en application les fondamentaux de la programmation en Python nécessaires à l'analyse de données et au développement de systèmes intelligents. L'organisation en dépôt GitHub permet d'assurer la traçabilité du code, la reproductibilité des résultats et la validation par les captures d'exécution.